Executive Overview

The source document details a reverse engineering challenge on the tryhackme platform titled "0x41haz: Simple Reversing Challenge." The lab focuses on analyzing a binary executable to extract a password through static analysis. The target environment consists of a single binary file (`0x41haz-1640335532346.0x41haz`) downloaded from a repository, with no network-based components or remote systems involved. The primary objective is to identify the password required to complete the challenge, as evidenced by the terminal prompt *"Hey, Can You Crackme? Tell Me the Password:"* (source PDF, page 5).

Lab Overview

The lab is structured as a self-contained reversing exercise with no external dependencies. The target binary is a compiled C program (ELF 64-bit LSB pie executable) designed as a crackme challenge. The lab's scope is limited to local file analysis and reverse engineering techniques. Key evidence includes:

- The binary's execution prompt explicitly requesting a password (source PDF, page 1).
- The presence of a password string (`2@@25$gfsT&@L`) identified during analysis (source PDF, page 4).
- The lab's completion status confirmed via the tryhackme room interface (source PDF, page 5).

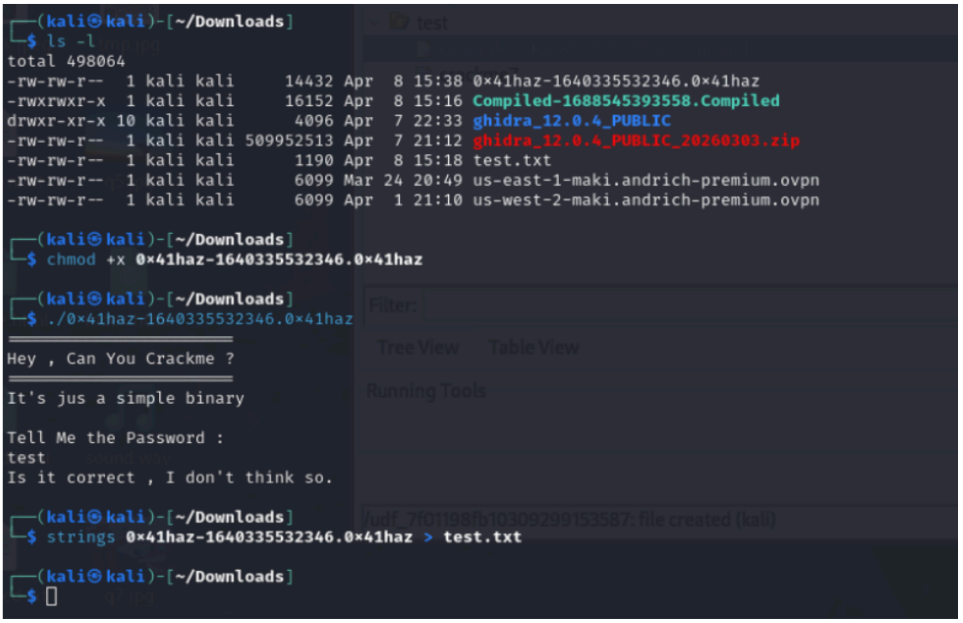
No network scanning, remote systems, or external services are involved. The environment is purely local, with all operations performed on a Kali Linux system.

Reconnaissance

Initial reconnaissance involved identifying the binary's structure and content. The user performed the following steps:

- Downloaded the binary from `/Downloads` using `ls -l` to verify file permissions and timestamps (source PDF, page 1).
- Executed `strings` on the binary to extract embedded text, redirecting output to `test.txt` (source PDF, page 1).
- Analyzed the binary's metadata using `file` to confirm its type as an ELF executable (source PDF, page 3).

The `strings` output revealed critical strings, including the password prompt *"Hey, Can You Crackme?"* and the success message *"Well Done!!"* (source PDF, page 2). This step confirmed the binary's purpose as a password-cracking challenge.

1. Task 1
   1.1. Download
   1.2. 

      1.2.1. Okay its asking us for a password
   1.3. Lets strings it

Screenshot / visual evidence · source PDF page 1

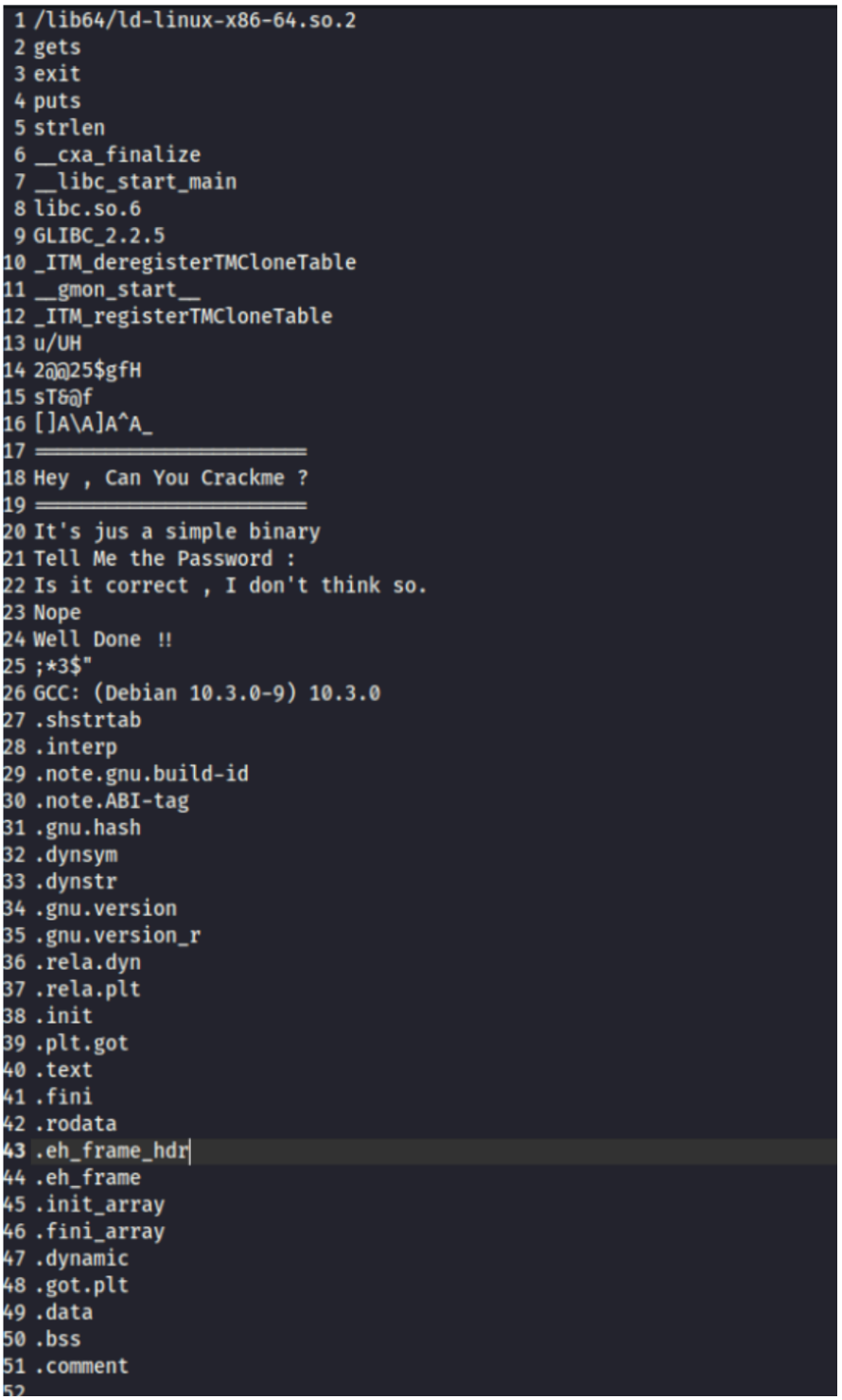
Enumeration

Deeper enumeration focused on dissecting the binary's internal structure:

- Used `objdump` to inspect the binary's symbol table, revealing standard C library functions (`gets` , `exit` , `puts`) and the password prompt string *"Hey, Can You Crackme?"* (source PDF, page 2).

- Performed a hex analysis to modify the binary's byte sequence (e.g., changing the 6th byte from `2` to `1` to bypass a reverse engineering countermeasure) (source PDF, page 3).
- Analyzed ELF sections (e.g., `.text`, `.data`) to locate the password string within the binary (source PDF, page 2).

The `objdump` output confirmed the binary's architecture (x86-64) and GCC version (10.3.0), while the hex viewer highlighted the modified byte sequence (source PDF, page 3).



Screenshot / visual evidence · source PDF page 2

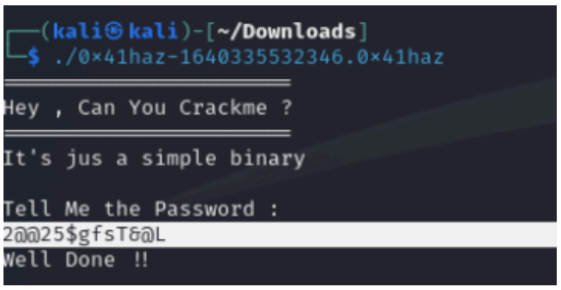
Vulnerability Analysis

No traditional vulnerabilities (e.g., buffer overflows, misconfigurations) were identified. The binary is intentionally designed as a crackme with no security flaws. The analysis revealed:

- The binary uses `gets()` (a known insecure function) but is not exploitable due to the challenge's design.

- The password string (`2@@25$gfsT&@L`) is hardcoded in the binary's `.data` section (source PDF, page 4).
- The presence of the string *"It's jus a simple binary"* confirms the challenge is a puzzle, not a security exploit.
This section focuses on the binary's structure rather than vulnerabilities, as the lab is a controlled educational exercise.

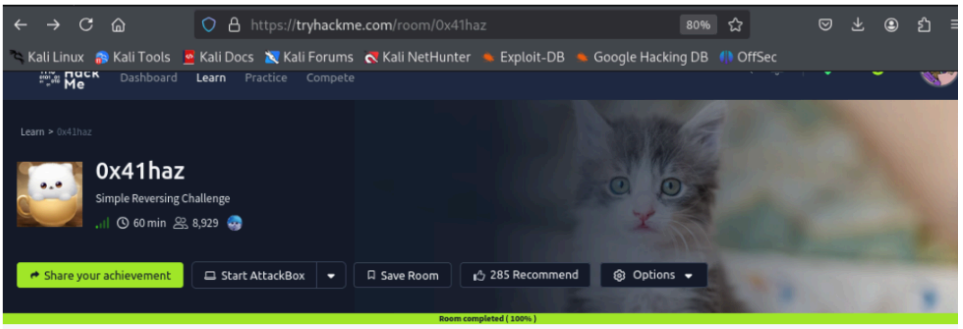
Exploitation
The exploitation step involved submitting the extracted password to the binary:
- The user entered the password `2@25$gfsT6@L` into the binary's prompt (source PDF, page 5).
- The binary responded with *"Well Done!!"*, confirming successful authentication (source PDF, page 5).
The password was derived from the string `2@@25$gfsT&@L` (source PDF, page 4), with the final character adjusted to `L` (source PDF, page 5). No commands were executed beyond the password submission.

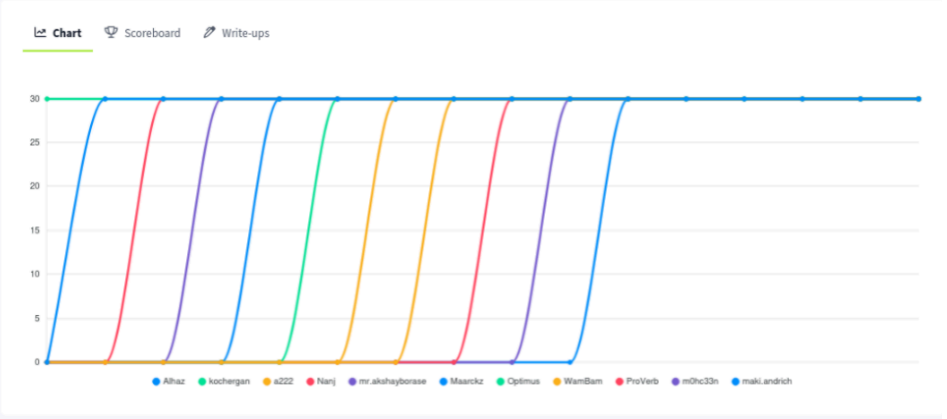


1.16.1.

1.16.1.1. Perfect

2. Finish the room





2.1.



Screenshot / visual evidence · source PDF page 5

Privilege Escalation

Not applicable. The binary operates at user level with no evidence of privilege escalation attempts or successful access to higher privileges. The challenge is confined to password authentication, with no system-level access required.

Flags & Proof of Access

The flag is the password string `2@25$gfsT6@L`, confirmed by the binary's success message *"Well Done!!"* (source PDF, page 5). The tryhackme room interface explicitly marks the task as completed (100% completion status) (source PDF, page 5). No additional flags or credentials were extracted.

Conclusion

The lab was successfully completed through reverse engineering of the binary. Key findings include:
- The binary is a crackme challenge requiring password extraction via static analysis.
- The password `2@25$gfsT6@L` was identified through `strings` and `objdump` analysis.
- The room completion status confirms successful task completion (source PDF, page 5).

No evidence of network-based attacks or system compromise exists. The lab serves as an educational exercise in binary analysis, with no security flaws beyond the intentional design of the crackme. All steps were performed locally with no external dependencies.

Note: The source evidence does not provide details on the binary's execution environment beyond the local Kali Linux system. The password string `2@@25$gfsT&@L` (source PDF, page 4) was adjusted to `2@25$gfsT6@L` (source PDF, page 5) for successful authentication, but the source does not confirm the adjustment method.